

Chapter 6 Basic Processor Design

6.1 Basic CPU Design Patterns

6.2 Start with a simplified instruction set

6.3 Components that make up a data path

6.4 Basic Data Path Design

6.5 More complete data path design

6.1 Basic CPU design patterns

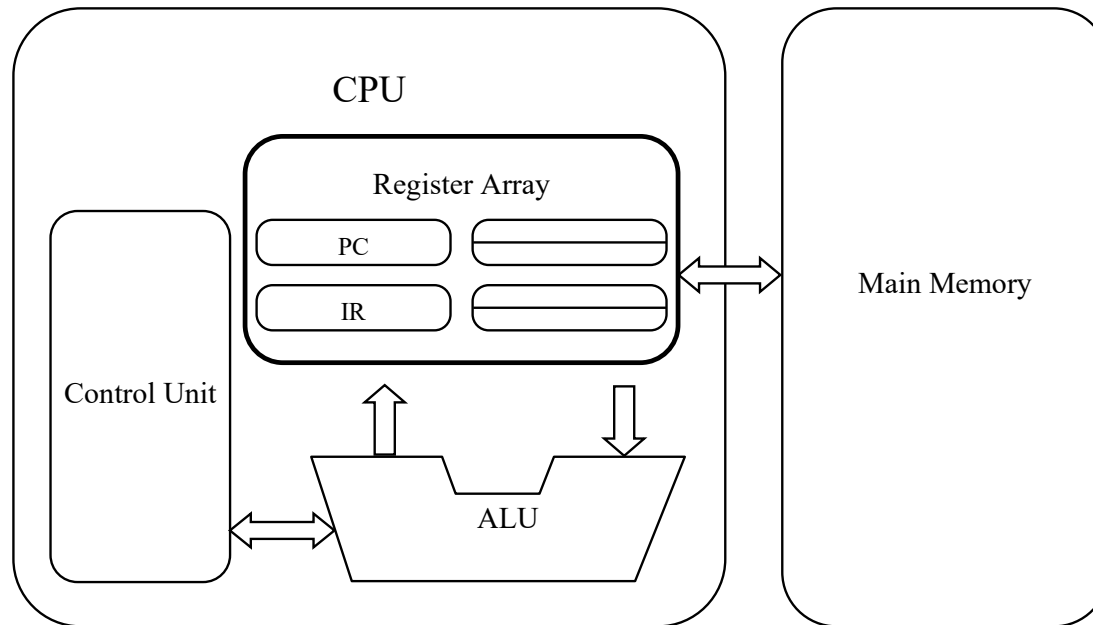
- ▶ Recognize the data path from the constituent components
- ▶ Look at the CPU structure from the command execution cycle

6.1.1 Recognize the data path from the constituent components

- ▶ Traditional saying :
 - ▶ The CPU is mainly composed of two parts: arithmetic/logic unit (ALU) and control unit (CU).
- ▶ Modern computers :
 - ▶ ALU 、 CU
 - ▶ register
 - ▶ Add other components that accelerate computing
 - ▶ floating-point processing unit(FPU)
 - ▶ Extended instruction processor

6.1.1 Recognize the data path from the constituent components

- The general architecture of modern CPUs :



6.1.1 Recognize the data path from the constituent components

► Datapath :

- Once all the execution instructions are properly integrated with the components needed to process the data, a complete data path is formed. (datapath)
- The most important task when designing a CPU is to structure a data path and complete the correct control.

6.1 Basic CPU design patterns

- ▶ Recognize the data path from the constituent components
- ▶ Look at the CPU structure from the command execution cycle

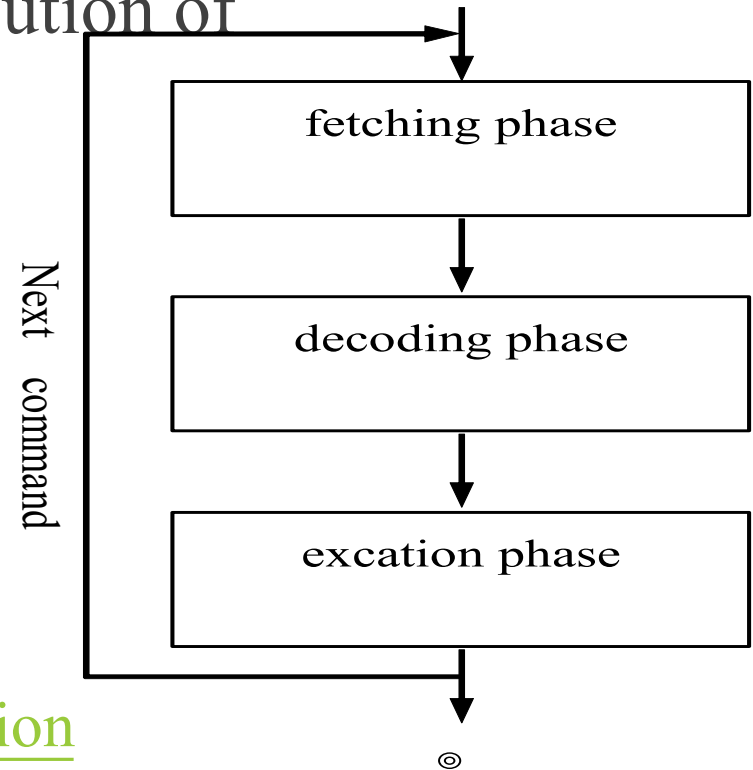
6.1.2 Look at the CPU structure from the command execution cycle

► The three phases of CPU execution of instructions :

- 1.fetching phase
- 2.decoding phase
- 3.execution phase

[cpu three phases](#)

[cpu phases step by step animation](#)



6.1.2 Look at the CPU structure from the command execution cycle

▶ Data path :

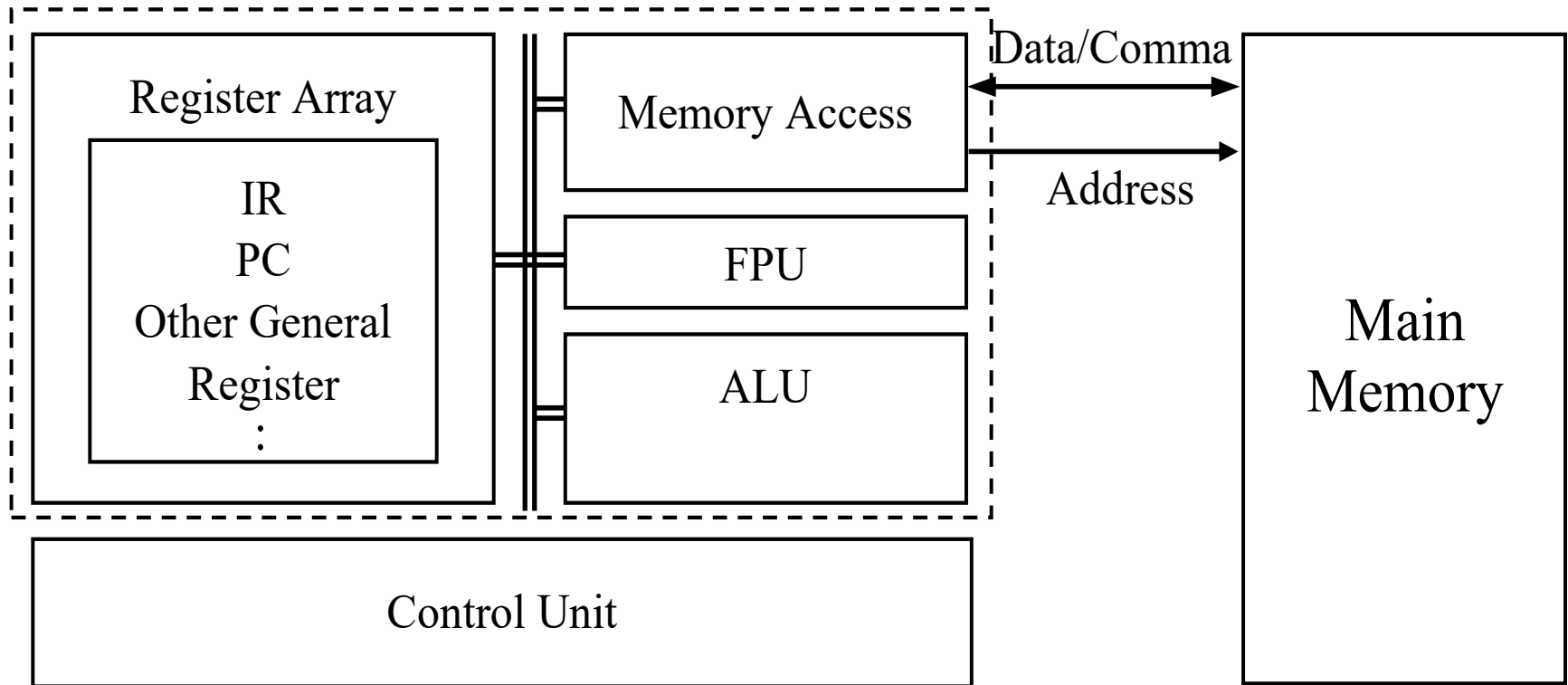
▶ The flow of the command execution :

- ▶ Data flows through several different components according to the type of instruction to receive the corresponding processing.
- ▶ Together, the channels interconnected between components are called data paths.

▶ control unit :

- ▶ Arrange data to enter the correct compute unit and start the components of the computation unit in a timely manner.

6.1.2 Look at the CPU structure from the command execution cycle



The CPU can be regarded as a combination of a data path and a control unit, and the data path is in the dotted box in the figure

6.1.2 Look at the CPU structure from the command execution cycle

► Fetching phase

- First, use a register dedicated to storing the next instruction address (commonly called a program counter, referred to as a **PC**) to retrieve the correct program instructions from memory to another specific register (usually called instruction register, referred to as **IR**); After the command is retrieved, the next command address to be retrieved will be automatically calculated and stored in the PC. Although this stage does not actually execute the instructions in the program, it is an indispensable step in the command execution process. And it actually involves data access between registers and memory, as well as arithmetic operations for PCs, so it can also be regarded as an action on the data path.

6.1.2 Look at the CPU structure from the command execution cycle

► Decoding phase

- This stage is almost entirely dependent on the control unit. Usually, the control unit will first take out the operation code (**op_code**) field in the command according to the definition of the instruction set and decode it. The remaining part of the instruction is then broken down into various addressing types of operatives (such as constants, register numbers, or direct/indirect address values) based on the decoded results, to obtain all the parameters of the command. In order to perform the operation correctly, the control unit must further decompose the instructions into one or several microinstructions that can be processed by the data path action based on the decoding results, and generate corresponding control signals to all computing units such as registers, memory, and arithmetic/logic. It sends the correct control signal to the control lines of each digital component, so that the execution of the command is completed in the intended form, content, and order.

6.1.2 Look at the CPU structure from the command execution cycle

► Execution phase

- After the decoding stage interprets all the messages in the command and takes out the parameters contained in the instructions, the next stage is to follow the clock drive and control code instructions to let the data enter one of the **specific processing units** inside the CPU, such as logic, arithmetic, bit shift, etc., for calculation, and finally the calculation results are stored in the appropriate registers or memory.

6.2 Start with a simplified set of instructions

- ▶ Necessary and basic operations for the processor :
 - ▶ Memory fetch
 - ▶ Addition/subtraction of integer
 - ▶ Logic operation
 - ▶ Bit Shift Operation
 - ▶ Branch/jump directives

6.2 Start with a simplified set of instructions

- ▶ Memory access command
- ▶ Integer addition and subtraction command
- ▶ Logical operation instructions
- ▶ Bit shift command
- ▶ Branch jump command

6.2.1 Memory access command

- ▶ Two types of memory access commands :
 - ▶ load word ◦
 - ▶ lw A, B
 - ▶ Reads the value of the character in the memory address stored in register B and stores it in register A.
 - ▶ Store word is stored in memory.
 - ▶ stw A, B
 - ▶ Saves the character value currently contained in register A to the memory address stored in register B.

memory access types

6.2.2 Integer addition and subtraction command

► Addition and subtraction instructions :

► Additive instructions :

► add A, B, C

► The sum of the values in the registers B and C is stored in register A, that is, $A=B+C$.

► Subtraction directive :

► sub A, B, C

► Subtract the value in register B from the value in register C, and store the difference in register A, i.e. $A=B-C$.

6.2.3 Logical operation instructions

- ▶ There are three commonly used logic instructions, which can be combined to create various logical operations :
 - ▶ AND command :
 - ▶ and A, B, C
 - ▶ The values stored in the B and C registers are performed bitwise and operation, and the results are stored in register A, that is, $A = B \text{ and } C$.
 - ▶ OR command :
 - ▶ or A, B, C
 - ▶ The values of B and C registers are performed bit by bit or operation, and the values are stored in register A, that is, $A = B \text{ or } C$.

6.2.3 Logical operation instructions

- ▶ XOR instructions :

- ▶ xor A, B, C

- ▶ perform exclusive or operation on the values of B and C registers bitwise, and store the values in **register A**, that is, **$A = B \text{ xor } C$** ; Basically, if the two source parameters of the xor command are the same variable, the result will be the inverse of the value, that is, **$A \text{ xor } A = \text{not } A$** .

6.2.4 Bit shift command

► Shift command :

- Move all bits "at the same time" in the "same direction" by n bits.
- It can be divided into the following three types :
 - logical shift
 - arithmetic shift
 - circular shift

6.2.4 Bit shift command

► logical shift

- Whether shifted to the left or right, the value of the bits that are moved out of the storage range will be discarded; The bit space vacated after the shift is always **filled with 0**.
- The command format is as follows :
 - `lsh A, n, d`
 - Logically shifts the values in register A n bits, and d represents the direction of shift, moving n bits to the left when d is 0, and n bits to the right if d is 1.

6.2.4 Bit shift command

► arithmetic shift :

- It is equivalent to multiplying or dividing the original value by the previous 2 index, moving one place to the left means multiplying by 2, and moving one place to the right means dividing by 2.
- The positive and negative symbols of the original value must be taken into account, and usually the calculator stores the negative value in the form of 2' complement.
- The command format is as follows :
 - ash A, n, d
 - The value in the A register is arithmetically shifted by n bits, d represents the direction of shift, when d is 0, it is moved n to the left, and the n bits on the far right of the register are always filled in 0; It should be noted that when the leftmost n+1 bits of the original value are not the same bit value, an overflow will occur and additional processing may be required. Conversely, if d is 1, all bits in the original value are moved n bits to the right, and the leftmost n bit space of the register is always filled with the same value as the original symbol bit.

6.2.4 Bit shift command

► Circular shift

- Treat the register space occupied by a variable as a circular continuous space, whether moved left or right, and insert it from the other end for each bit moved.
- The command format is as follows :
 - `csh A, n, d`
 - Move the value in the A register n bits in a cycle, d represents the direction of shift, when d is 0, move n bits to the left, and after the value of the original leftmost n bits is removed, the rightmost n bit space of the register is filled again in the original order; If d is 1, the original value is moved n bits to the right, and the value of the original right n bits is moved out, and the leftmost n bit space of the register is re-filled in the original order.

6.2.4 Bit shift command

- The result of a 32-bit number shifted 4 bits to the right through three shift types :

Original input word group	1010 1101 0011 0111 0000 0101 0011 1110
Perform logic right shift	0000 1010 1101 0011 0111 0000 0101 0011
Perform arithmetic right shift	1111 1010 1101 0011 0111 0000 0101 0011
Perform a circular right shift	1110 1010 1101 0011 0111 0000 0101 0011

6.2.5 Branch jump command

▶ branch

- ▶ Conditional changes in the execution order.
- ▶ Flags can be used to control branches.
- ▶ It can be used with the Compare command.

▶ jump

- ▶ "Unconditionally" jumps to a specific command address to start execution.

Branch and Jump Instructions with Examples

6.2.5 Branch jump command

► Compare Commands :

► comp A, B

- Comparing the sizes of A and B values is actually equivalent to performing A-B operations. When the two numbers are equal, set the **zero flag** value to 1, otherwise set to 0;

If $A < B$, set the value of the negative flag to 1,
otherwise set to 0.

These two flag values are of considerable importance for branching or not.

6.2.5 Branch jump command

► branch instructions :

► brz C

- When the value of the zero flag is equal to 1, the order of command execution is changed, and the next command address to be executed is the value stored in register C;

However, if the **zero flag** value is not 0, do not take any further action and continue to execute the command in the address stored on the current PC.

► brless C

- When the value of the negative flag is equal to 1, the order of command execution is changed, and the next command address to be executed is the value stored in register C; However, if the negative flag value is not 0, do not take any further action and continue to execute the command in the address stored on the current PC.

6.2.5 Branch jump command

- ▶ Jump command :

- ▶ `jp A`

- ▶ Regardless of the execution result of the previous command, the next command address to be executed is the value stored in register A.

6.3 The components that make up the data path

- ▶ **Decoder vs Multiplexer**
- ▶ register array
- ▶ Addition/subtraction machine
- ▶ logical computing unit
- ▶ Shift operator

Classic x86 Data Path 1995 2005

PC Data Path Final Fixed

6.3.1 Decoder vs Multiplexer

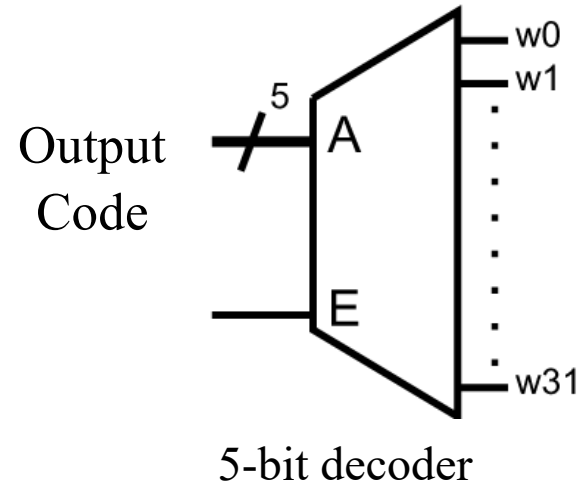
► decoder

► input :

- A binary string of bits.

► output :

- The output signal is equivalent to the numerical magnitude that the input can represent.
- Important components that control or select paths.
- When used as a switch, only one switch will be turned on at a time, and all the rest will be off.



6.3.1 Decoder vs Multiplexer

► multiplexor

► input :

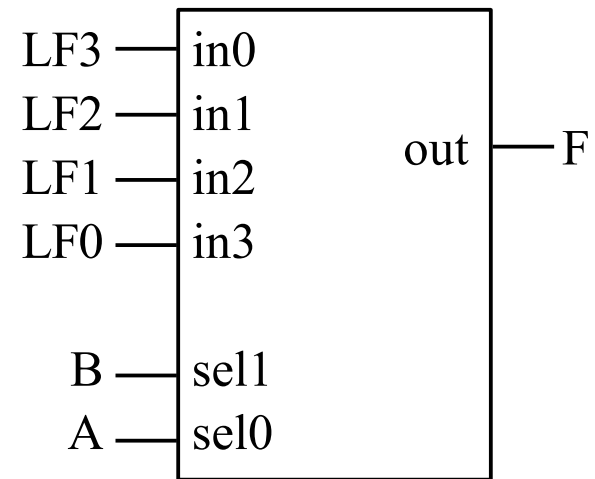
- A binary string.
- The input relative to the maximum number of binary bits that can be represented.

► output :

- One of all the data passed in by the input side.

► function :

- Use a set of codes to select one of the values from multiple values and output them.



4-to-1 multitasker

6.3 The components that make up the data path

- ▶ Decoder vs Multiplexer
- ▶ Register array
- ▶ Addition/subtraction machine
- ▶ logical computing unit
- ▶ Shift operator

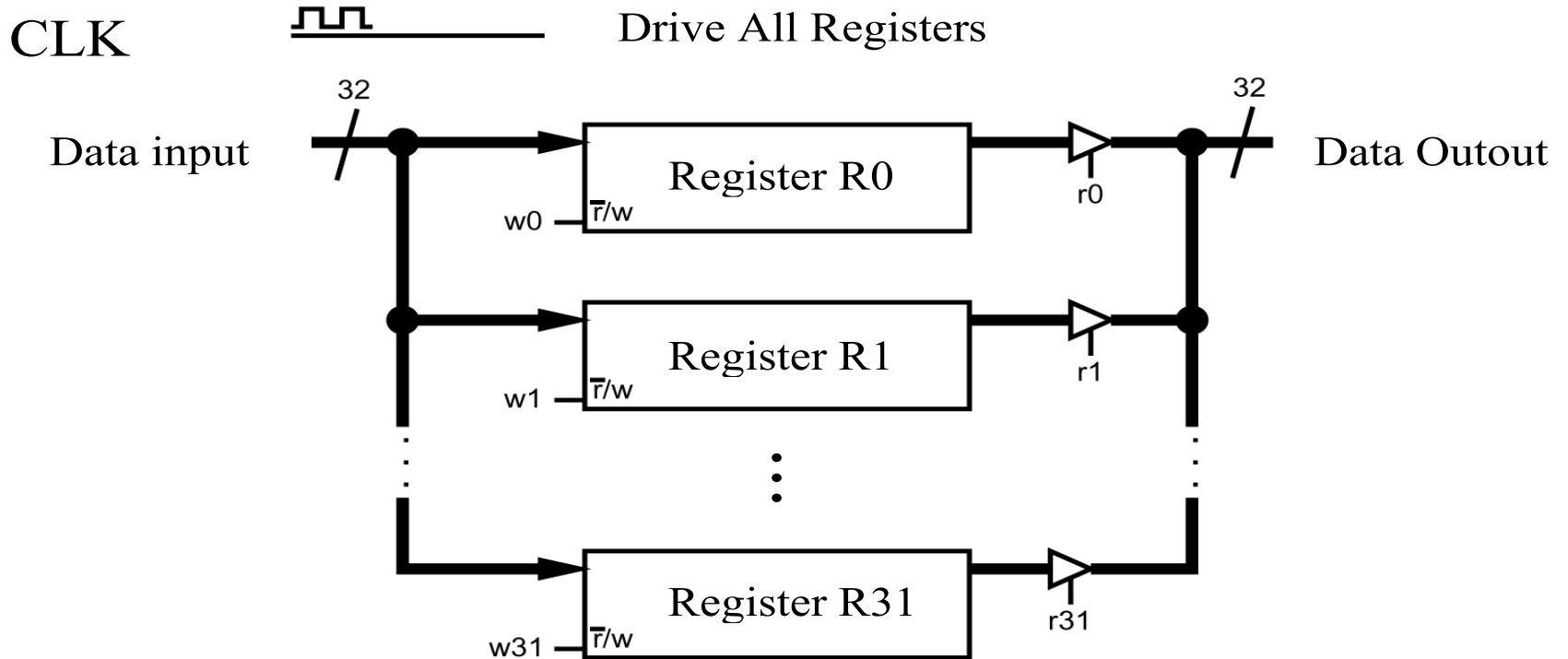
6.3.2 Register array

- ▶ There will be a considerable number of registers in computers, which can be divided into :
 - ▶ special purpose registers
 - ▶ Program counter (PC), instruction register (IR).
 - ▶ general purpose registers
 - ▶ The program is free to use by the user.

6.3.2 Register array

- ▶ Register array
 - ▶ Registers operate as arrays.
 - ▶ Use index values to access individual registers.
 - ▶ index : The address of each register (address).
 - ▶ Also known as register files (register file) ◦
 - ▶ Addressable (addressable)

6.3.2 Register array



The internal structure of a register array

6.3.2 Register array

- ▶ All registers, whether written or read, are written or read through a common input and output bus.
- ▶ In practice, it is not possible to write to all registers at once, or read the contents of all registers at the same time.
- ▶ Determining which register to write to or from which register to output data is determined by two 32-to-1 decoders, "write" and "read", so that only one register read/write on a bus will be triggered at a time.

6.3.2 Register array

► Register array architecture :

► input :

► XA 、YA : 5-bit index value (address).

► ZA : 5-bit index value that specifies the register to be written.

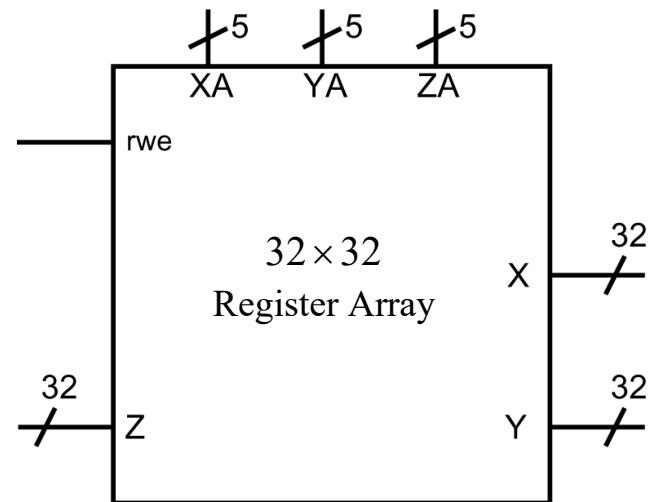
► Z : Write the value of the register.

► output :

► X,Y : The value read out.

► control :

► rwe : Writes are allowed (register write enable) .



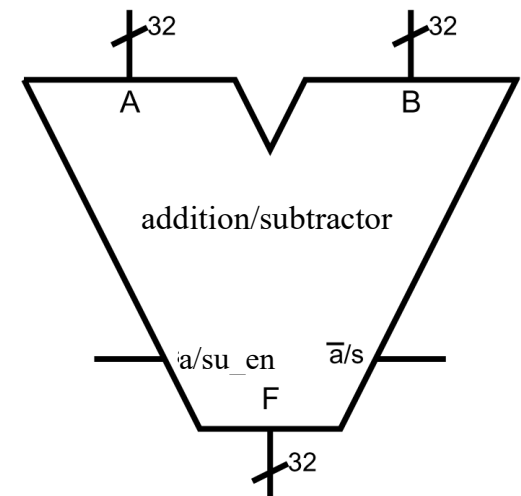
Register array components
are viewed from the outside

6.3 The components that make up the data path

- ▶ Decoder vs Multiplexer
- ▶ Register array
- ▶ Addition/subtraction machine
- ▶ logical computing unit
- ▶ Shift operator

6.3.3 Addition/subtraction machine

- ▶ 32 An integer addition/subtractor for bits (adder/subtractor)
 - ▶ input :
 - ▶ The two inputs A and B can each input a 32-bit integer.
 - ▶ output :
 - ▶ The F-end outputs a 32-bit operation result, which is $A+B$ or $A-B$.
 - ▶ Control side :
 - ▶ Activate the addition/subtractor of the a/su_en (adder/subtractor unit enable) °
 - ▶ Decide on the $\bar{a}/s$ side of the operation to be added or subtracted.



32-bit add/subtractor components

6.3 The components that make up the data path

- ▶ Decoder vs Multiplexer
- ▶ Register array
- ▶ Addition/subtraction machine
- ▶ Logical computing unit
- ▶ Shift operator

6.3.4 Logical computing unit

► Logic unit

- Responsible for logical operations.

 - and 、 or 、 xor

- It is often built with multiplexers, and each multiplexer can complete a single bit of operation at a time.

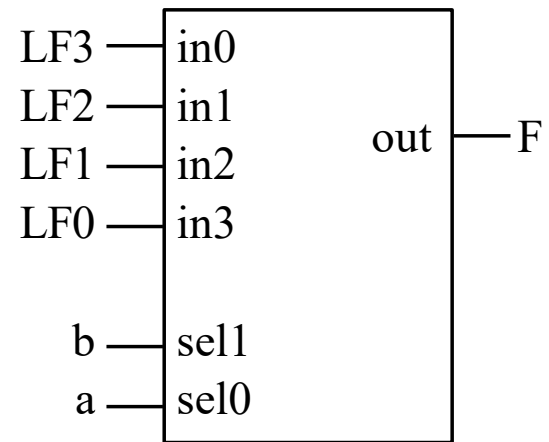
6.3.4 Logical computing unit

- The truth table is used to represent the logical functions of and, or, and xor respectively.

b	a	Logical functions	and	or	xor
0	0	LF_0	0	0	0
0	1	LF_1	0	1	1
1	0	LF_2	0	1	1
1	1	LF_3	1	1	0

6.3.4 Logical computing unit

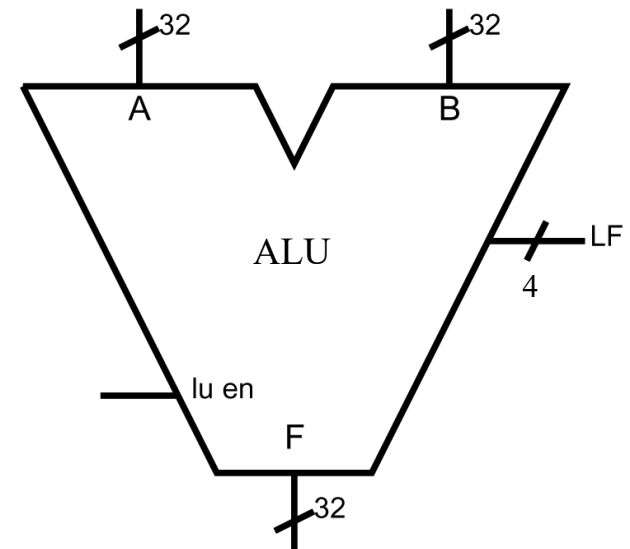
- ▶ Single-bit logical operator
 - ▶ After the truth table is created, it is implemented using a multiplexer.
 - ▶ import :
 - ▶ a and b.
 - ▶ export :
 - ▶ The F-value represents the result of the operation, and the multiplexer selects the corresponding bits in the logical function based on the combination of b/a inputs.
 - ▶ control :
 - ▶ Logical functions LF3 LF2 LF1 LF0 ◦



4-to-1 multiplexers implement single-bit logical operators

6.3.4 Logical computing unit

- ▶ 32-bit logic operator
 - ▶ 32 unit-element logic operators are combined and driven by a common clock.



32-bit logic operator

6.3 The components that make up the data path

- ▶ Decoder vs Multiplexer
- ▶ Register array
- ▶ Addition/subtraction machine
- ▶ logical computing unit
- ▶ Shift operator

6.3.5 Shift operator

- ▶ Progressive shift computer :
 - ▶ All bits move in the direction specified by the command at the same time, moving only one bit at a time.
 - ▶ To shift n bits, it must be done n times.
 - ▶ There is no efficiency, additional mechanisms that require control of the number of times.

6.3.5 Shift operator

► Progressive **left** shift process :

Original input word group	<u>0110 1101</u> 0011 0111 0000 0101 0011 1110
First shift	1101 1010 0110 1110 0000 1010 0111 110x
Second shift	1011 0100 1101 1100 0001 0100 1111 10xx
The third shift	0110 1001 1011 1000 0010 1001 1111 0xxx
Fourth shift	1101 0011 0111 0000 0101 0011 1110 xxxx
Fifth shift	1010 0011 0111 0000 0101 0011 111x xxxx
Sixth shift	0100 1101 1100 0001 0100 1111 10xx xxxx
Seventh shift	1001 1011 1000 0010 1001 1111 0xxx xxxx
Eighth shift	0011 0111 0000 0101 0011 1110 xxxx xxxx
Ninth shift	0110 1110 0000 1010 0111 110x xxxx xxxx

6.3.5 Shift operator

► barrel shifter

► It can perform shift operations of any number of bits between 0 and 31 bits in a single clock.

► It is connected by 5 smaller shifters, that is, it is shifted in 5 times 2^k .

(k from 4 to 0)

► Combining the shiftable digits in binary form exactly represents all numbers between 0 and 31.

6.3.5 Shift operator

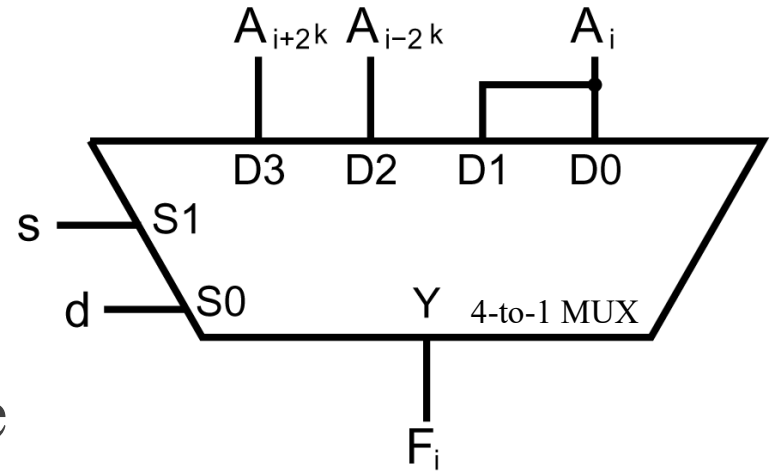
► The multi-bit shifter moves 9 bit instances :

Original input word group	0110 1101 0011 0111 0000 0101 0011 1110
Shift left by 2^k k=4 bits (0)	0110 1101 0011 0111 0000 0101 0011 1110
Shift left by 2^k k=3 bits (1)	0011 0111 0000 0101 0011 1110 xxxx xxxx
Shift left by 2^k k=2 bits (0)	0011 0111 0000 0101 0011 1110 xxxx xxxx
Shift left by 2^k k=1 bits (0)	0011 0111 0000 0101 0011 1110 xxxx xxxx
Shift left by 2^k k=0 bits (1)	0110 1110 0000 1010 0111 110x xxxx xxxx

2^k k=4 as (2 to the power of 4)

6.3.5 Shift operator

- The shifter in 2^k each stage can be implemented with 32 multiplexers, and the result of the 2^k shift of the leftmost or rightmost bit will be determined by the type and direction of the shift.

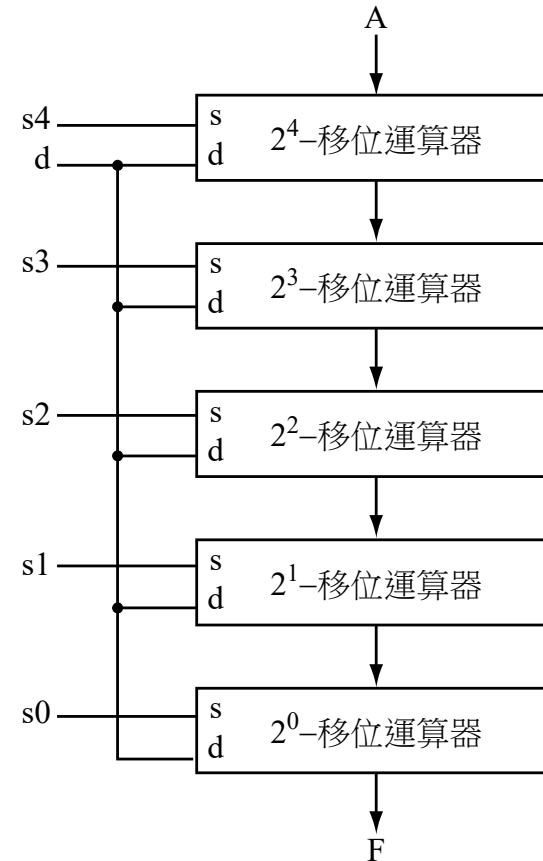


A single bit of each shifter 2^k

6.3.5 Shift operator

► 32-bit shifter :

- Connect the 5 shifters 2^k in series and add inputs such as the shift direction and the number of shift bits in binary.



The main structure of the multi-position shifter

6.3.5 Shift operator

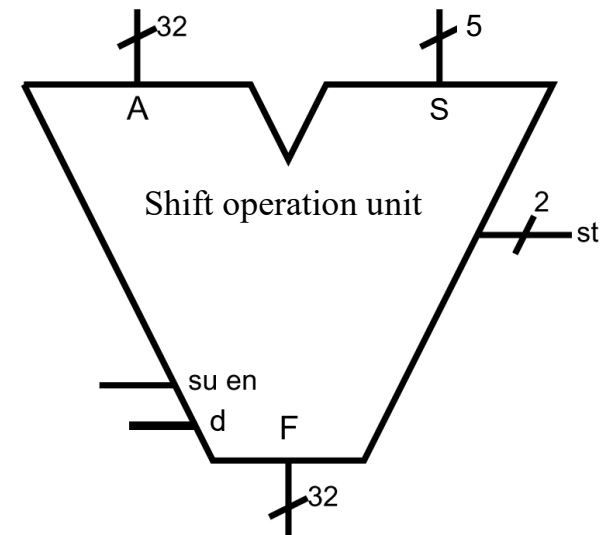
► A shifter set in the CPU :

► input :

- Switch of the shifter: Expressed in `su_en`, used to decide whether to activate the shifter.
- Word to be shifted: represented by `A`, which is the original value of the word.
- The direction to be shifted: is represented by `d`.
- Number of bits to shift: Expressed in `S`, a 5-bit unnumbered integer.
- Type of shift: Expressed in `st`, indicating what kind of shift operation is to be performed.
- Logical shift, arithmetic shift, cyclic shift.

► output :

- It is represented by `F`, which is the 32-bit character after the shift.



A shifter set in the data path

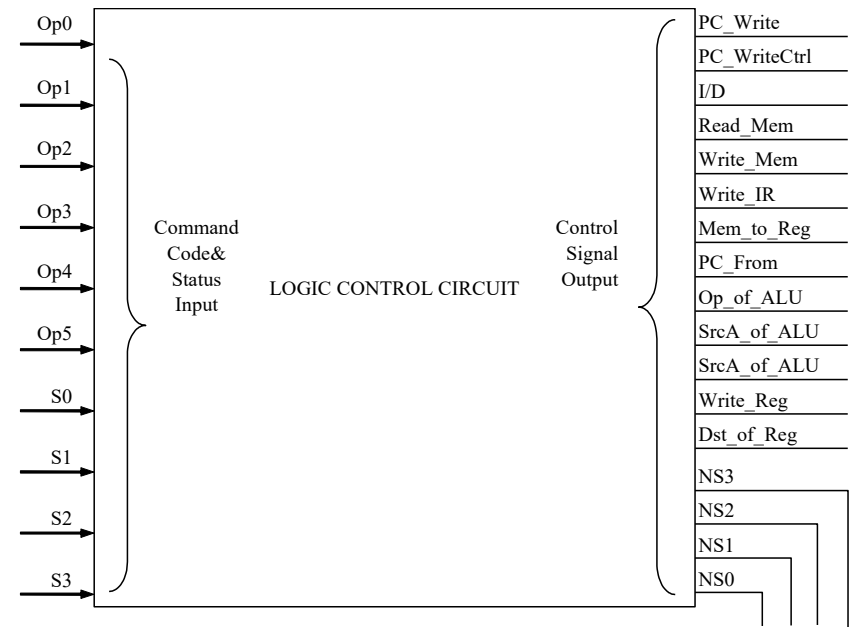
6.4 Basic data path design

- ▶ How to control a single-clock cycle data path
- ▶ Basic data path architecture
- ▶ Control of the basic data path
- ▶ Example of a basic data path microcommand

6.4.1 How to control a single-clock cycle data path

► Control unit hardware design :

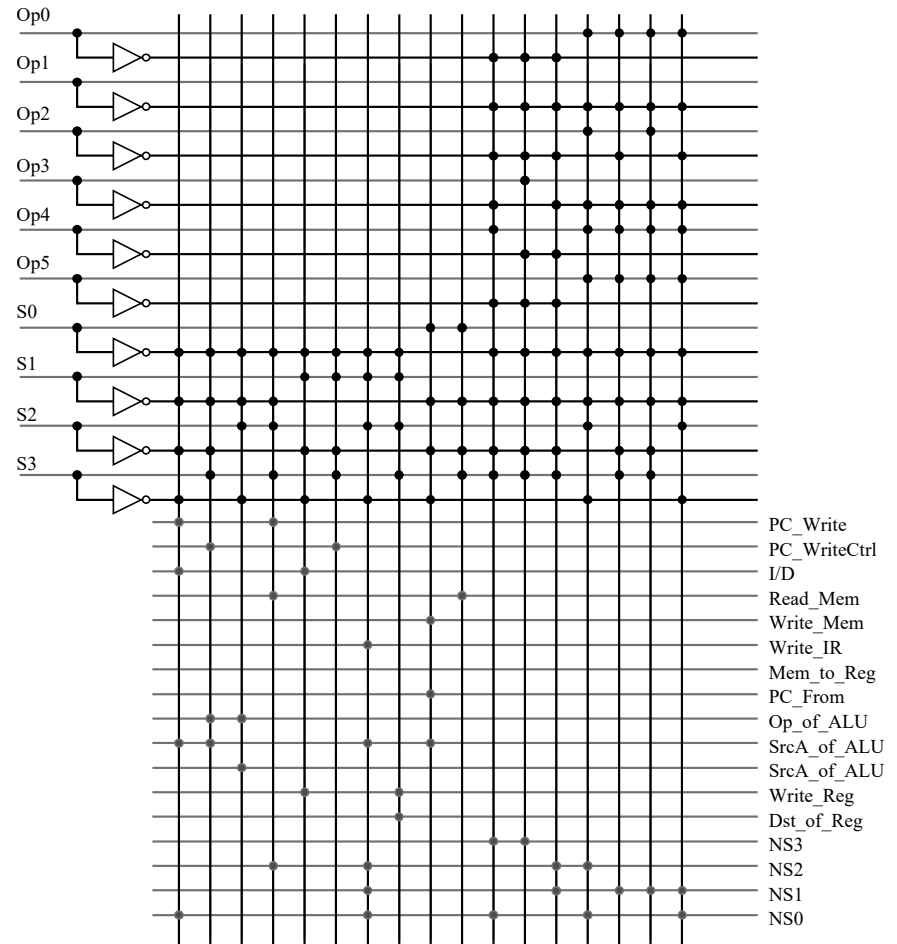
- Treat the physical control line as the decoded output of the actual machine command.
- It is called hardware wiring control (hardwired control)
- advantage :
 - Fast execution.
- disadvantage :
 - Circuits are complex and difficult to design or modify.
 - The cost of modification is high.



Hardware-wired control unit

6.4.1 How to control a single-clock cycle data path

- ▶ Control unit hardware design :
 - ▶ The Programming Logic Array (PLA) circuit can be used to complete the hardware design of the control unit of a single-clock processor.
 - ▶ Reduce costs.

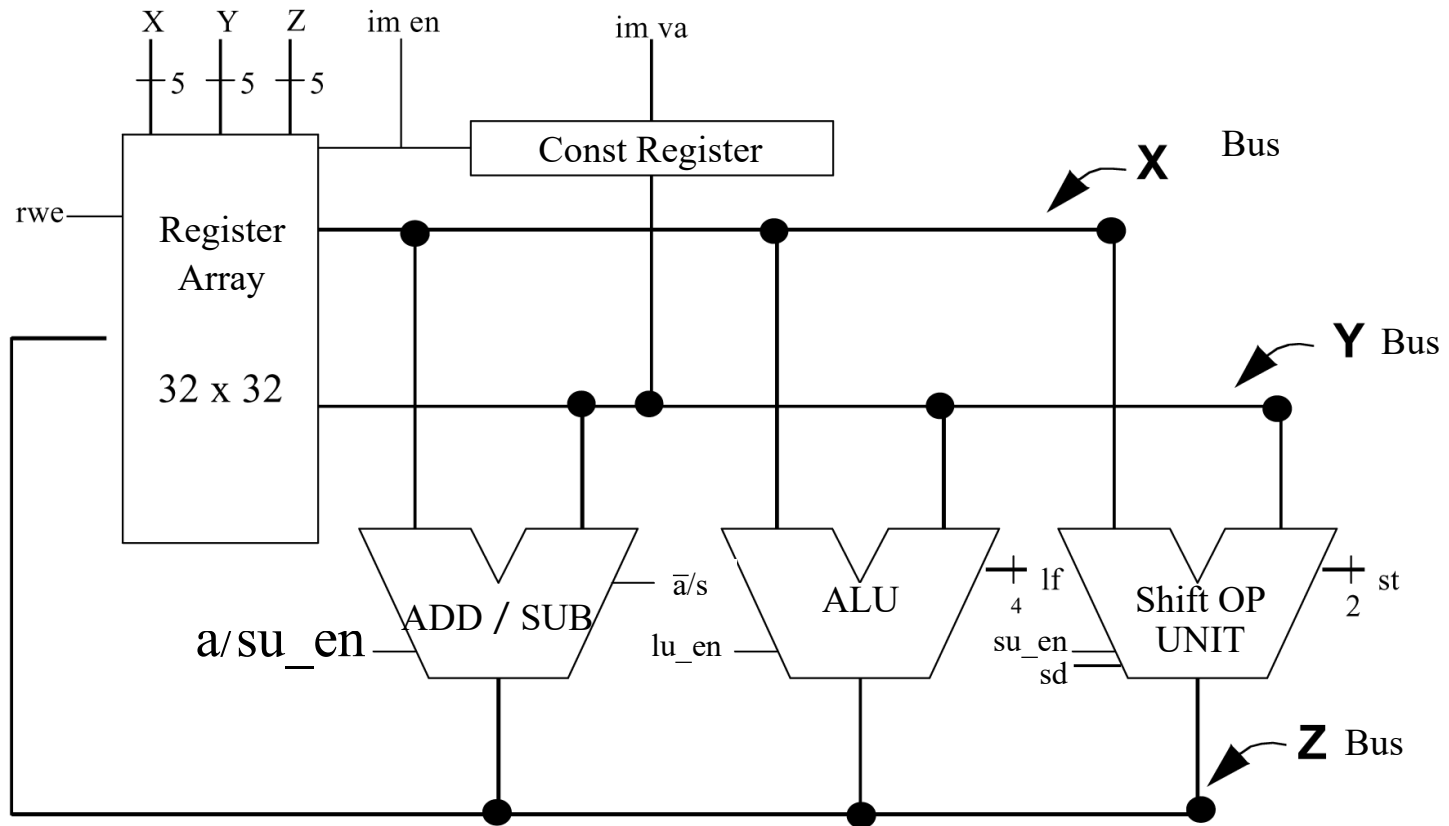


The control unit is implemented with a programmable logic array circuit

6.4 Basic data path design

- ▶ How to control a single-clock cycle data path
- ▶ **Basic data path architecture**
- ▶ Control of the basic data path
- ▶ Example of a basic data path microcommand

6.4.2 Basic data path architecture



The most basic single clock cycle data path

6.4 Basic data path design

- ▶ How to control a single-clock cycle data path
- ▶ Basic data path architecture
- ▶ **Control of the basic data path**
- ▶ Example of a basic data path microcommand

6.4.3 Control of the basic data path

► Control unit :

- A collection of lines that emit various control signals based on instructions to enable each computing unit to function correctly.
- To understand how the data path should be controlled, we must first look at the controllable or input side on the entire data path.

6.4.3 Control of the basic data path

► Register :

► register array address :

- Two source register addresses XA, YA

- A destination register address ZA

- The length is 5 bits.

► register array write switch :

- **rwe**, A 1-bit control signal is used to determine whether the result value of the calculation should be returned to the register array.

6.4.3 Control of the basic data path

► register :

► Constant register switch :

- **im_en**, A 1-bit control signal is used to determine the data source on the wire Y, when it is 0, the data on the second readout of the register array is used; If it is 1, the data in the constant register is used instead.

► Constant register data :

- **im_va** is a constant data with a length of 32 bits, which is used to input into a constant register to participate in the operation; This data does not count as a control signal, and when the command is decoded, the decoding line takes the constant out of the instruction and puts it into this register.

6.4.3 Control of the basic data path

- ▶ Addition/subtraction machine :
 - ▶ Addition/subtractor switch :
 - ▶ a/su_en , A 1-bit control signal used to decide whether to use an addition/subtractor or not.
 - ▶ Add/subtraction switching :
 - $\bar{a}/s$, when it is 0, it is added, and when it is 1, it is subtracted.

6.4.3 Control of the basic data path

- ▶ Logic operator :

- ▶ Logic computator switch :

- ▶ **lu_en**, A 1-bit control signal used to decide whether to use a logic operator.

- ▶ Logical functions :

- ▶ **Lf**, is a 4-bit signal, which represents the corresponding output of two input bits in pairs in the input data in four cases.

6.4.3 Control of the basic data path

► Shift operator :

► Shift compound switch :

- **su_en**, a 1-bit control signal used to decide whether to use the shift operator ◦

► The direction to be shifted :

- **sd**, 1 bit of control signal, shifted to the left when it is 0; shift to the right when it is 1 ◦

► type of shift :

- **st**, a 2-bit control signal, when it is 00, it means that logical shift should be performed, 01 means arithmetic shift should be performed, and when it is 10, cyclic shift should be performed.

6.4.3 Control of the basic data path

► CPU internal microinstructions :

XA	YA	ZA	rwe	imm_en	im_va	a/su_en	$\bar{a}/s$	lu_en	LF	su_en	sd	st
5	5	5	1	1	32	1	1	1	4	1	1	2

Instruction-controlled strings

6.4 Basic data path design

- ▶ How to control a single-clock cycle data path
- ▶ Basic data path architecture
- ▶ Control of the basic data path
- ▶ Example of a basic data path microcommand

6.4.4 Example of a basic data path microcommand

► Example 1: Additive basic instruction operation

add R1, R2, R3

Convert this addition into microinstructions :

XA	YA	ZA	rwe	imm_en	im_va	a/su_en	$\bar{a}/s$	lu_en	LF	su_en	sd	st
2	3	1	1	0	x	1	0	0	xxxx	0	x	xx

6.4.4 Example of a basic data path microcommand

► Example 2: Basic logic operation instructions or R1, R2, R3

Convert this 'or' (or) operation into microinstructions :

XA	YA	ZA	rwe	imm_en	im_va	a/su_en	$\bar{a}/s$	lu_en	LF	su_en	sd	st
2	3	1	1	0	x	0	x	1	1110	0	x	xx

6.4.4 Example of a basic data path micro command

► Example 3: A more complex arithmetic example

$$A = (B + C + D) / 2 - (E + F)$$

- First, registers are used to map each variable in the arithmetic expression:
- $A = R1, B = R2, C = R3, D = R4, E = R5, F = R6$
- Next, the original formula is disassembled and represented by multiple consecutive basic instructions, and a register that stores the transition result is added in between 7:
- - Ad R1, R2, R3; $R1 = B + C$
 - Ad R1, R1, R4; $R1 = B + C + D$
 - Ash R1, 1, 1; $R1 = (B + C + D) / 2$
 - Ad R7, R5, R6; $R7 = A + F$
 - Sub R1, R1, R7; $R1 = (B + C + D) / 2 - (A + F)$

6.4.4 Example of a basic data path micro command

- ▶ **Example 3:** A more complex arithmetic example
- ▶ Finally, all instructions are converted to micro commands :

XA	YA	ZA	rew	imm_en	im_va	a/su_en	ā/s	lu_en	LF	su_en	sd	st
2	3	1	1	0	X	1	0	0	xxxx	0	x	xx
1	4	1	1	0	X	1	0	0	xxxx	0	x	xx
1	x	1	1	1	1	0	x	0	xxxx	1	1	01
5	6	7	1	0	X	1	0	0	xxxx	0	x	xx
1	7	1	1	0	X	1	1	0	xxxx	0	x	xx

6.4.4 Example of a basic data path micro command

► Example 4: Logical combinatorial operation

$$A = (B \text{ or } C) \text{ xor } (D \text{ and } E)$$

► Map all variables to registers first :

$$A = R1, B = R2, C = R3, D = R4, E = R5$$

► Next, the original formula is disassembled and represented by multiple consecutive basic instructions :

or R1, R2, R3 ;R1=B or C

and R6, R4, R5 ;R6=D and E

xor R1, R1, R6 ;A=R1=(B or C) xor (D and E)

6.4.4 Example of a basic data path micro command

- **Example 4:** Logical combinatorial operation
- Finally, all instructions are converted to micro commands :

XA	YA	ZA	rew	imm_en	im_va	a/su_en	$\bar{a}/s$	lu_en	LF	su_en	sd	st
2	3	1	1	0	X	0	x	1	1110	0	x	xx
4	5	6	1	0	X	0	x	1	1000	0	x	xx
1	6	1	1	0	X	0	x	1	0110	0	x	xx

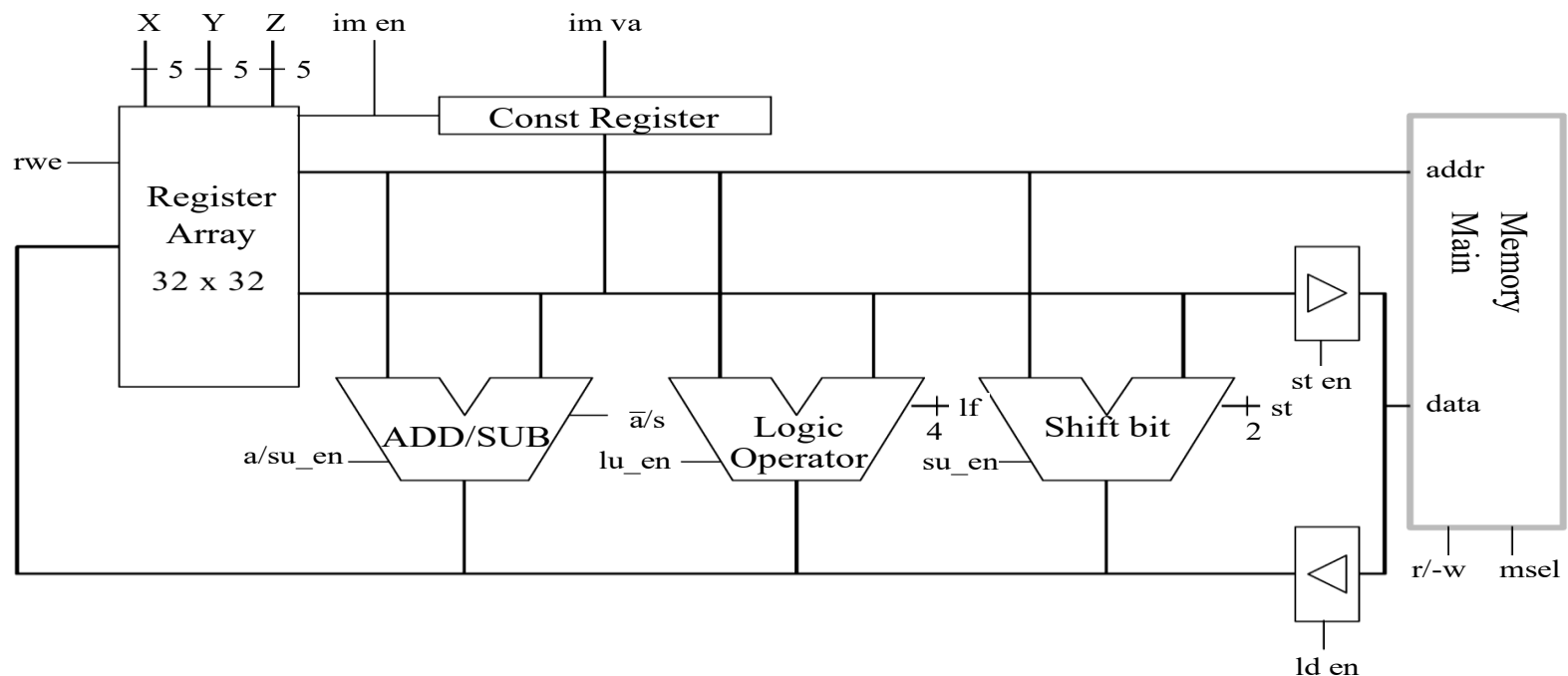
6.5 More complete data path design

- ▶ Connect the memory module
- ▶ Contains the data path for the command ingestion phase

6.5.1 Connect the memory module

► Changes in architecture and controls

- Data access to memory can also be done through three rows of X, Y, and Z, so it can be directly connected as a component on the data path.



6.5.1 Connect the memory module

- ▶ memory controller :
 - ▶ Switch of memory modules :
 - ▶ In **MSEL**, 1 means memory is accessible, and 0 is inaccessible.
 - ▶ Read/write selection :
 - ▶ expressed as **r/-w**, 1 means that the value can only be read from memory, and 0 can only be written; However, when msel=0, r/-w is meaningless and can be ignored.

6.5.1 Connect the memory module

- ▶ Memory controller :

- ▶ Write back :

- ▶ expressed in `st_en`, if it is 1, it means that the value on the Y line can be written to memory; This value cannot be 1 at the same time as the following allowable load value to avoid errors ◦

- ▶ Allow loading :

- ▶ Expressed in `ld_en`, if it is 1, it means that the value read from memory can be passed in to the Z line and uploaded back; This value cannot be 1 at the same time as the above allowable writeback value to **avoid errors**.

6.5.1 Connect the memory module

- After adding memory, the new microinstruction framework :

XA	YA	ZA	rew	imm_en	im_va	a/su_en	$\bar{a}/s$	lu_en	LF	su_en	sd	st	mselect	r/ $\bar{w}$	st_en	ld_en
5	5	5	1	1	32	1	1	1	4	1	1	2	1	1	1	1

After adding memory, the command controls the string of bits

6.5.1 Connect the memory module

- **Example 1:** Write data to the memory space of the location address

lw R1, R2

- This is the most basic loading instruction, which will treat the value in register R2 as the memory address to be read, and pass it to memory through the X input line to read the value in that address; The returned value is stored in register R1.

XA	YA	ZA	rew	imm_en	im_va	a/su_en	ā/s	lu_en	LF	su_en	sd	st	mselect	r/ŵ	st_en	ld_en
2	X	1	1	0	X	0	X	0	xxxx	0	x	xx	1	1	0	1

6.5.1 Connect the memory module

► Example 2: Unconditional jumping

jp R1

- Basically, this command does not really involve memory access, but only changes the next command address to be retrieved, that is, changes the value of the PC.

XA	YA	ZA	rew	imm_en	im_va	a/su_en	ā/s	lu_en	LF	su_en	sd	st	mselect	r/w	st_en	ld_en
1	X	31	1	1	0	0	X	1	1110	0	x	xx	0	x	0	0

6.5.1 Connect the memory module

- **Example 3:** Provide combinatorial arithmetic for variable addresses

$$M[R1] = (M[R2] + R3 + M[R4]) * 3 - M[R5]$$

- Expand the above with the command in the command set to get it :

```
ld R6, R2
add R6, R6, R3
ld R7, R4
add R6, R6, R7
add R8, R6, 0
ash R8, 1, 0
add R8, R6, R8
ld R9, R5
sub R10, R8, R9
stw R10, R1
```

```
;R6=M[R2]
;R6= M[R2]+R3
;R7=M[R4]
;R6= M[R2]+R3+M[R4]
;R8= M[R2]+R3+M[R4]
;R8= (M[R2]+R3+M[R4])*2
;R8= (M[R2]+R3+M[R4])*3
;R9=M[R5]
;R10=(M[R2]+R3+M[R4])*3 - M[R5]
;M[R1]=(M[R2]+R3+M[R4])*3 - M[R5]
```

6.5.1 Connect the memory module

- Example 3: Provide combinatorial arithmetic for variable addresses

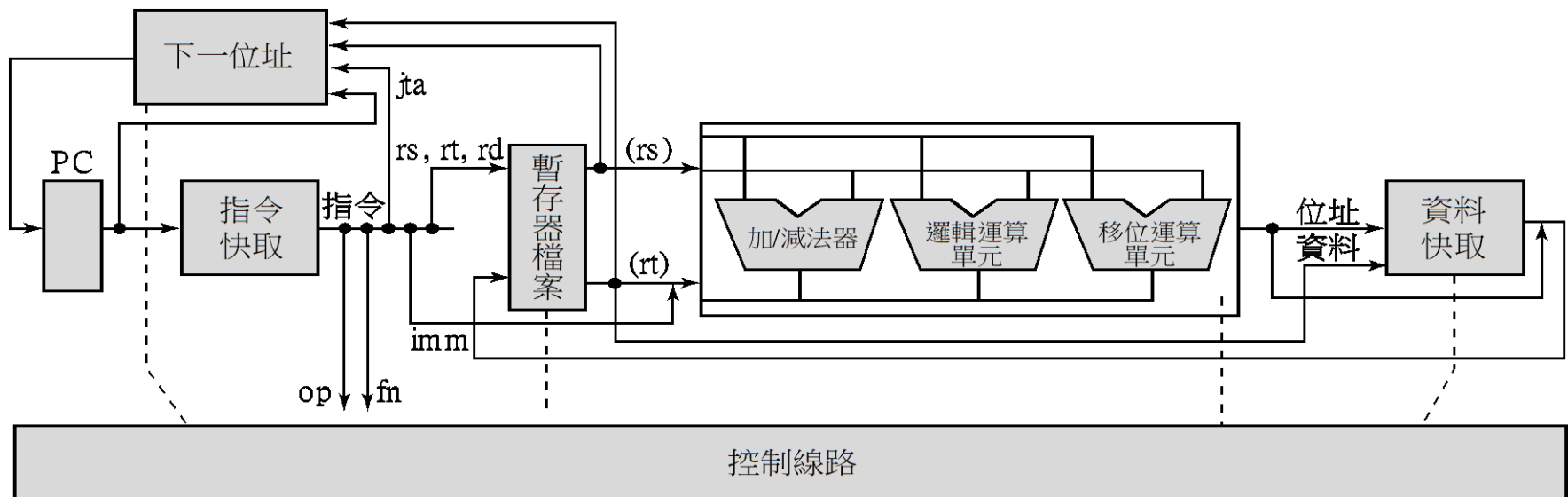
XA	YA	ZA	rew	imm_en	im_va	a/su_en	ā/s	lu_en	LF	su_en	sd	st	msel	r/ŵ	st_en	ld_en
2	X	6	1	0	X	0	X	0	xxxx	0	x	xx	1	1	0	1
6	3	6	1	0	X	1	0	0	xxxx	0	x	xx	0	x	0	0
4	X	7	1	0	X	0	X	0	xxxx	0	x	xx	1	1	0	1
6	7	6	1	0	X	1	0	0	xxxx	0	x	xx	0	x	0	0
6	X	8	1	1	0	1	0	0	xxxx	0	x	xx	0	x	0	0
8	X	8	1	1	1	0	X	0	xxxx	1	0	01	0	x	0	0
6	8	8	1	0	X	1	0	0	X	0	x	xx	0	x	0	0
5	X	9	1	0	X	0	X	0	X	0	x	xx	1	1	0	1
8	9	10	1	0	X	1	1	0	X	0	x	xx	0	x	0	0
1	X	10	1	0	X	0	X	0	X	0	x	xx	1	1	1	0

6.5 More complete data path design

- ▶ Connect the memory module
- ▶ Contains the data path for the command ingestion phase

6.5.2 Contains the data path for the command ingestion phase

- Commands are also part of the data, so data paths can be used to calculate the address to maintain the correct execution flow.



After adding the calculations for the PC, the defined instruction set can be fully implemented

The End